



Phase 5: Query Optimization and Indexing Report

Queries Chosen and Performance Issues Observed

For this lab, I tested 5 queries on our online_shop database. These queries included customer sales totals, monthly sales reports, top-selling products, customer order history, and pending orders.

Initially, all queries were slow because no indexes existed on the tables. Query 1 took 82 seconds to complete because it had to scan the entire Customer and Order tables. Query 5 took 34 seconds because the status column had no index, forcing a full table scan. The EXPLAIN command showed key = NULL and type = ALL meaning full table scans were happening.

Optimization Strategies Implemented

I added indexes on columns used in WHERE, JOIN, and ORDER BY clauses. Specifically, I created indexes on customer_id in both Customer and Order tables, product_id in Product and OrderItem tables, and status in the Order table. After adding indexes, I re-ran EXPLAIN to check if they were being used.

Improvements Achieved

The results were mixed but educational. Query 1 improved dramatically from 82 seconds to 1.27 seconds which is a 98% improvement. The customer_id index was used effectively here.

However, Query 2 slowed down from 1.14 seconds to 5.44 seconds. Query 3 slowed from 3.73 to 6.02 seconds. Query 4 slowed terribly from 3.73 seconds to 187.58 seconds. Query 5 slowed from 34.52 seconds to 171.45 seconds.

This shows that adding indexes without careful analysis can actually harm performance. MariaDB chose different execution plans after indexing which caused some queries to become slower.

Recommendations for Future Query Writing

Based on my experience, here are my recommendations for large-scale systems:

First, always run EXPLAIN before and after adding indexes to see if they are being used. Second, add indexes one at a time and test performance after each one. Third, avoid adding too many indexes because each index slows down INSERT, UPDATE, and DELETE operations. Fourth, use covering indexes when possible so the database can get all needed data from the index alone without reading the actual table. Fifth, regularly review slow queries as data grows because what works at 10,000 rows may fail at 1 million rows.

The most important lesson I learned is that optimization requires testing. You cannot assume indexes will always help. Each query must be analyzed individually and tested with real data volumes.